

Quão Forte é o seu Validator? Medindo Portões de Aceitação para Programas Sintetizados por LLM

Carlos Eduardo Dias Batista
Pesquisador independente
Brasil

Abstract

Architectures that let an LLM synthesize disposable programs and accept their outputs through an immutable validator are only as strong as the validator. A previous study of such an architecture (IDAE) showed that format-level validators let well-formed but wrong outputs through, and that one faulty program then corrupts every record of its format. This paper asks how to build a stronger validator and how to measure its strength before deployment. We assemble a corpus of 1,068 unique programs actually synthesized by Claude Opus 4.5 and Claude Haiku 4.5 for three data-extraction domains with a per-record ground-truth oracle; 70 of them are defective, with 23 distinct faulty behaviors (misread number conventions, time-zone and epoch errors, guessed values for missing fields, fabricated fields under an unsatisfiable specification). We compare format validators, source-anchored validators, property validators that relate the output to the raw input, metamorphic validators, N-version agreement, and 80 validators written by the LLMs themselves. Property validators written from the specification alone, without access to the corpus, caught 21 of the 23 faulty behaviors (format validators: 6), rejecting 0.4% of correct outputs. N-version agreement caught 78–100% but rejected 8–27% of correct outputs. LLM-written validators traded silent corruption for false rejection: only 1 of 10 Opus 4.5 and 0 of 10 Haiku 4.5 validators kept false rejection at or below 1% in the financial domain. Letting the same models revise their validators for up to five rounds against automatic self-checks on the specification's examples did not close the gap: in the financial and IoT domains, 4 of 40 validators were usable before and 4 of 40 after, because validators tuned to one example per format still rejected correct records the examples did not cover. A gate that requires each new program to reproduce a few verified input–output pairs of its format never rejected a correct program and caught exactly the faults the property validators missed; combined, they caught 96–100% of the faulty behaviors. A cheap mutation score correlated with strength against real faults (Spearman $\rho = 0.51$, 95% CI [0.36, 0.66], 100 validators), useful as a screen but not as a substitute. Measured against 14 consumption tasks fixed before the analysis, 20 of the 23 faulty behaviors changed the result of at least one task, and the format validator inverted the priorities: half of the faults it caught were the only three harmless ones, and every fault it let through changed some task. The two faults that escaped the property validators were a plausible re-conversion and an ambiguity in the specification itself, which no validator derived from it can resolve, but a verified example can.

Keywords

LLM, Program Synthesis, Test Oracle, Validator, Metamorphic Testing, N-Version Programming, Mutation Testing, Silent Data Corruption

Resumo

Arquiteturas em que um LLM sintetiza programas descartáveis e um validator imutável decide quais saídas aceitar são tão fortes quanto o validator. Um estudo anterior de uma dessas arquiteturas (IDAE) mostrou que validators de formato deixam passar saídas bem-formadas porém erradas, e que um único programa defeituoso então corrompe todos os registros do seu formato. Este artigo pergunta como construir um validator mais forte e como medir sua força antes de colocá-lo em produção. Montamos um corpus com 1.068 programas únicos realmente sintetizados por Claude Opus 4.5 e Haiku 4.5 em três domínios com oráculo por registro; 70 são defeituosos, com 23 comportamentos defeituosos distintos. Validators de propriedades escritos só a partir da especificação pegaram 21 dos 23 (os de formato, 6), rejeitando 0,4% das saídas corretas. N-version pegou 78–100%, mas rejeitou 8–27% das saídas corretas. Validators escritos pelos próprios LLMs trocaram corrupção silenciosa por rejeição falsa, e deixar o LLM revisá-los em até cinco rodadas de autoverificação contra os exemplos da especificação não mudou isso: nos domínios financeiro e IoT, 4 de 40 eram utilizáveis antes e 4 de 40 depois. Um portão que exige que cada programa novo reproduza poucos pares entrada–saída verificados do seu formato não barrou nenhum programa correto e pegou justamente os defeitos que os validators de propriedades deixaram passar; juntos, pegaram de 96% a 100%. O escore de mutação correlacionou com a força contra falhas reais ($\rho = 0,51$), útil como triagem, mas não como substituto. Medidas contra 14 tarefas de consumo fixadas antes da análise, 20 dos 23 comportamentos defeituosos mudaram o resultado de ao menos uma tarefa, e o validator de formato inverteu as prioridades: metade do que ele pegou eram os únicos três defeitos inofensivos, e tudo o que ele deixou passar mudou alguma tarefa.

Palavras-chave: LLM, síntese de programas, oráculo de teste, validator, teste metamórfico, programação N-version, teste de mutação, corrupção silenciosa de dados.

DOI: <https://doi.org/10.5281/zenodo.23074363> (todas as versões)

Versão em inglês: <https://doi.org/10.5281/zenodo.23074532>

Artefatos: <https://github.com/carloseduardodb/idaae-validators>

1 Introdução

Uma forma cada vez mais comum de usar LLMs em engenharia de software é pedir ao modelo um *programa* em vez de pedir a *resposta*: o programa é gerado uma vez, verificado e reutilizado. O

IDAE (*Intent-Driven Adaptive Extraction*) [3] leva essa ideia à extração de dados: para cada formato de entrada, um LLM sintetiza um programa descartável; um validator derivado de uma especificação imutável (o *telos*) decide se cada saída pode ser aceita; o programa aprovado é reutilizado para todos os registros do mesmo formato. O custo cai em duas ordens de grandeza em relação a chamar o LLM por registro.

A avaliação do IDAE com um oráculo de verdade-terreno por registro mostrou o preço dessa economia: **a garantia do sistema é exatamente a força do seu validator**. Um validator que verifica tipos, faixas e formatos, como o validator do telos no IDAE, não sabe se um valor bem-formatado está certo. Quando um programa lê "3440.04" como número brasileiro e entrega 344004, o validator aceita, e a amortização repete o erro em todos os registros daquele formato. É o problema do oráculo em teste de software [2], agora no caminho crítico da produção.

Este artigo trata a pergunta seguinte: **como construir um validator para programas sintetizados por LLM, e como medir, antes do deploy, quanto ele garante?** Respondemos seis questões de pesquisa:

- **RQ1 (força):** quanto das falhas que LLMs realmente cometem cada estratégia de validator deixa passar, e quantas saídas corretas ela barra?
- **RQ2 (medida barata):** o escore de mutação de um validator, obtido sem LLM e sem oráculo de produção, prevê sua força contra falhas reais?
- **RQ3 (validator escrito pelo LLM):** um validator escrito pelo próprio LLM a partir da especificação é tão bom quanto um escrito com cuidado? Melhora se o LLM puder revisá-lo com autoverificação? Os erros do validator e do programa se correlacionam quando vêm do mesmo modelo?
- **RQ4 (custo):** quanto cada estratégia custa em tempo de verificação, execuções extras e chamadas de LLM?
- **RQ5 (pares verificados):** se cada formato tiver alguns pares entrada-saída verificados, e o formato não mudar durante o uso, quanto um portão que exige reproduzir esses pares acrescenta a um validator?
- **RQ6 (impacto na tarefa):** os defeitos que cada validator deixa passar mudam o resultado das tarefas que consomem as saídas, ou alguns são inofensivos para quem usa os dados?

1.1 Contribuições

- (1) **Um corpus de falhas reais** de programas sintetizados por LLM: 1.068 programas únicos de Claude Opus 4.5 e Haiku 4.5, reexecutados contra um oráculo por registro, com 23 comportamentos defeituosos distintos, organizados numa taxonomia (Seções 3.2 e 4.1).
- (2) **Uma comparação de seis estratégias de validator** com a mesma régua (formato, ancorado na fonte, propriedades, metamórfico, N-version [1] e escrito por LLM), medindo tanto corrupção silenciosa quanto rejeição falsa.

- (3) **Evidência de que relações entrada-saída derivadas da especificação** pegam 21 de 23 comportamentos defeituosos com 0,4% de rejeição falsa, escritas sem ver o corpus e congeladas antes da avaliação.
- (4) **Evidência de que validators escritos pelo LLM** são fortes, mas raramente utilizáveis, por rejeitarem saídas corretas, e de que revisá-los com autoverificação contra os exemplos da especificação não resolve o problema.
- (5) **Evidência de que pares verificados e relações entrada-saída são complementares:** poucos pares verificados por formato pegam os defeitos que as relações não distinguem, as relações pegam os que só aparecem em casos de borda, e juntos pegam de 96% a 100% dos comportamentos defeituosos sem barrar nenhum programa correto.
- (6) **Uma avaliação do escore de mutação como triagem** da força de um validator, com correlação moderada e positiva contra falhas reais.
- (7) **Uma medida do impacto dos defeitos nas tarefas de consumo**, com tarefas e tolerâncias fixadas antes da análise: quase todo defeito muda alguma tarefa, e o validator de formato pega justamente os inofensivos.

2 Contexto

2.1 O validator como portão

No IDAE, um programa σ sintetizado para um formato é aplicado a cada registro x daquele formato. A saída $\sigma(x)$ só é aceita se $V(\sigma(x), x) = \text{True}$; se o validator barra a saída, o programa é descartado e outro é sintetizado. Um programa também pode devolver null, rejeitando o registro por conta própria. Chamamos de *corrupção silenciosa* uma saída aceita e diferente da verdade (ou aceita quando o registro deveria ser rejeitado), e de *rejeição falsa* uma saída correta barrada. A corrupção silenciosa se multiplica pela amortização; a rejeição falsa custa uma nova síntese e, no limite, esgota o orçamento de tentativas e rejeita um formato inteiro.

2.2 Estratégias de validator

Formato (V0). Verifica tipos, faixas e formatos da saída, como um contrato [18]. É o validator do telos no IDAE.

Ancorado na fonte (V1). V0 mais a exigência de que o identificador da saída apareça literalmente na entrada. Proposto no IDAE contra identificadores inventados.

Propriedades (V2). V0 mais relações entre a saída e a entrada bruta, derivadas da especificação: cada número da saída deve ser explicável por um número da entrada sob as conversões permitidas; campos derivados devem ser coerentes; datas devem corresponder a datas presentes na entrada; se a especificação manda rejeitar quando falta um dado, a entrada precisa conter evidência desse dado. É a ideia de teste baseado em propriedades [7] aplicada em produção, sobre cada saída.

Metamórfico (V3). V0 mais relações metamórficas [4, 20]: o validator reexecuta o mesmo programa em entradas transformadas (identificador trocado, valor dobrado, data deslocada) e verifica se a saída muda como a especificação implica.

N-version (V4). V0 mais concordância com um segundo programa sintetizado independentemente para o mesmo formato [1,

19]. A premissa de independência de falhas é conhecida por ser frágil [16].

Escrito por LLM (L). O próprio LLM recebe a especificação e exemplos e escreve o validator, como em abordagens que geram testes ou juizes com o próprio modelo [5, 22].

3 Desenho do Estudo

3.1 Domínios e oráculo

Usamos os três domínios do IDAE: financeiro (transações em quatro moedas, com conversão para USD, data UTC e categoria), IoT (telemetria com conversão de unidades, timestamp UTC e status derivado) e 560 eventos sísmicos reais do USGS, com dois telos: o *corrigido* e o *estrito*, que exige um campo ausente no CSV do USGS e, portanto, obriga a rejeitar esses registros. Cada registro carrega a verdade-terreno: a saída exata esperada ou “deve ser rejeitado”.

Aos seis formatos de cada domínio sintético do IDAE acrescentamos cinco fases novas: chave=valor com símbolo de moeda e fuso variado, JSON aninhado com epoch, XML, JSON com valor e unidade no mesmo campo, e uma fase adversarial nesses formatos (7 variantes a rejeitar e 3 aceitáveis porém ardilosas). Cada fase nova tem assinatura de formato própria e não altera os registros das fases originais. O fluxo de avaliação usa a mesma semente do IDAE (1001), com 100 registros por fase e 50 nas adversariais.

3.2 Corpus de programas sintetizados

O corpus vem de duas fontes: (i) os logs JSONL do IDAE, com todas as respostas de síntese de programa sob o telos final (1.570 respostas), e (ii) uma rodada nova com o mesmo prompt de síntese do IDAE, copiado sem alterações, sobre os formatos antigos e novos: três amostras por assinatura de formato, uma por prompt, para cada modelo (Claude Opus 4.5 e Haiku 4.5), e quatro amostras por formato do USGS nos dois telos (182 chamadas, nenhuma falha). As amostras da rodada nova vêm de um fluxo com semente diferente da avaliação (3001), para que nenhum programa seja avaliado no registro que viu.

Cada resposta é normalizada (comentários e espaços removidos) e deduplicada por domínio, assinatura e hash do código. Cada programa é reexecutado, isolado e com limite de tempo, sobre todos os registros da sua assinatura, que são os registros aos quais o IDAE o aplicaria por amortização. O resultado por registro, independente de qualquer validator, é *certo*, *errado* (saída diferente da verdade, ou saída quando deveria rejeitar) ou *falta* (null ou erro quando deveria aceitar). Um programa é *defeituoso* se produz ao menos uma saída errada. Como muitos programas textualmente diferentes eram exatamente nos mesmos registros, agrupamos programas por *comportamento* (o vetor de resultados por registro e os campos errados) e reportamos os resultados principais por comportamento distinto.

A Tabela 1 resume o corpus. Dos 1.068 programas únicos, todos compilam; 70 são defeituosos e formam 23 comportamentos defeituosos distintos. O Haiku 4.5 produziu 25 programas defeituosos em 106; o Opus 4.5, 45 em 962. Por comportamento, 13 dos 23 são só do Haiku, 7 só do Opus e 3 aparecem nos dois.

Tabela 1: Corpus de programas sintetizados (programas únicos por domínio e rótulo contra o oráculo).

Domínio	Progr.	Corretos	Incompl.	Defeit.
Financeiro	549	474	27	48
IoT	476	457	10	9
USGS corrigido	22	20	0	2
USGS estrito	21	10	0	11
Total	1.068	961	37	70
comport. distintos		27	15	23

3.3 Validators avaliados

V0 e V1 são os validators do IDAE, sem alterações. V2 e V3 foram escritos para este estudo **sem acesso ao corpus**: uma instância de um assistente de IA (Claude Opus 5.5, modelo mais recente que os estudados), isolada numa pasta com apenas a especificação de cada domínio, um exemplo *normal* de entrada por formato e o código de V0, escreveu os dois validators sob restrições explícitas: tratar a entrada como uma cadeia opaca (sem depender de nomes de campos, delimitadores ou posições de coluna dos exemplos, já que novos formatos aparecem), não reimplementar a transformação, e nunca rejeitar uma saída correta dos exemplos. A instância derivou à mão a saída correta de cada exemplo e escreveu um script de autoverificação (109 asserções) contra o qual iterou. O código foi congelado com hash SHA-256 no controle de versão antes da primeira execução contra o corpus. O autor e o assistente principal já tinham visto os defeitos do corpus quando V2 e V3 foram encomendados; por isso a escrita foi delegada a uma instância sem esse contexto.

V4 usa como parceiro outro programa do corpus com o mesmo domínio, a mesma assinatura e o mesmo modelo; como o resultado depende de qual parceiro é sorteado, reportamos três parceiros espalhados pelo corpus (V4-1 a V4-3). Os validators escritos por LLM (L) são 80: dez implementações independentes por modelo e domínio, pedidas com o mesmo material que a instância de V2 recebeu (especificação e um exemplo normal por formato) e as mesmas restrições, numa única chamada cada; todos compilaram e foram congelados antes da avaliação.

Autoverificação iterativa (I). Para separar o efeito do processo do efeito do modelo, cada um dos 80 validators L foi revisado pelo mesmo modelo que o escreveu, em até cinco rodadas no total. A cada rodada o validator é executado contra os mesmos exemplos do prompt: a saída correta de cada exemplo (a mesma informação que a instância de V2 derivou à mão) precisa ser aceita, e variantes erradas dessas saídas deveriam ser barradas. As variantes vêm de um mutador genérico por tipo de campo, sem nada do corpus: números $\times 10$, $\div 10$ e $+1$; datas ± 1 dia; timestamps $+1$ h e -1 dia; textos trocados por outro valor do mesmo campo nos exemplos. O modelo recebe o código atual, as saídas corretas rejeitadas, até seis variantes aceitas e eventuais erros, e devolve uma versão revisada. O processo para quando não há falhas; a versão final é a de menos rejeições falsas e, em seguida, menos variantes aceitas na autoverificação. V2 passa nessa autoverificação aceitando todas as saídas corretas e só 1 das 216 variantes. As versões finais também foram congeladas com hash antes da avaliação.

3.4 Métricas

Cada validator decide, para cada saída objeto de cada programa, aceitar ou barrar (a rejeição do próprio programa não chega ao validator). Um programa defeituoso é *pego* se o validator barra ao menos uma de suas saídas erradas (no IDAE, isso descarta o programa e dispara nova síntese) e *limpo* se todas as suas saídas erradas são barradas. Reportamos ainda a fração de saídas erradas barradas e a *rejeição falsa*: a fração de saídas corretas barradas.

3.5 Mutação

Para a RQ2, aplicamos nove operadores de mutação de primeira ordem ao código de até seis programas corretos por assinatura: literal numérico $\times 10$ e $+1$, troca de operador aritmético, limite de comparação, getUTC* trocado por hora local, remoção de normalização de texto, remoção de uma guarda de rejeição (o programa passa a “chutar”) e inversão de igualdade [14]. Dos 3.525 mutantes de 141 programas, 2.335 não mudaram o resultado, 419 só passaram a rejeitar ou quebrar e 506 repetiram um comportamento já visto; restaram 265 mutantes com comportamento distinto que produzem ao menos uma saída errada. O escore de mutação de um validator é a fração desses mutantes que ele pega. A pergunta é se esse escore, que não exige falhas reais, ordena os validators como as falhas reais ordenam [15].

3.6 Pares verificados

Para a RQ5, supomos que cada formato (assinatura) tem k registros cuja saída foi verificada (ou cuja rejeição foi confirmada) e que o formato não muda durante a janela de uso. Um programa só é aprovado se reproduz exatamente a verdade em todos os k pares: a mesma saída nos pares de aceitação e null nos de rejeição. Se falha, é descartado antes de processar qualquer registro. A verdade dos pares vem do oráculo, ou seja, supomos verificação perfeita. Comparamos três composições do conjunto: os k primeiros registros do formato, na ordem do fluxo (os que o programa veria primeiro em produção); um *sorteio* de k registros do formato, com probabilidade exata (hipergeométrica); e um conjunto *estratificado*, com um registro de cada tipo presente no formato (normal e cada caso de borda, inclusive os que devem ser rejeitados), completado ao acaso, com a média de 500 sorteios. Cada composição é avaliada sozinha (G) e combinada com V0 e com V2: o programa é pego se falha no portão ou se o validator barra ao menos uma de suas saídas erradas.

3.7 Tarefas de consumo

Para a RQ6, definimos o que um consumidor faria com as saídas de cada domínio: 14 tarefas, cada uma com uma tolerância declarada (Tabela 8). No financeiro: total em USD por mês e por categoria (erro relativo de até 1%), fração de transações internacionais (até 1 ponto percentual), dez maiores transações (ao menos 9 de 10 em comum) e conciliação diária (diferença de até 0,011 USD por transação do dia). No IoT: média por métrica, zona e mês (até 0,5 unidade), alertas críticos por métrica e mês (contagem exata), distribuição de pares métrica–status (até 2 pontos percentuais) e a série de alertas críticos por dispositivo e minuto (conjunto idêntico). No USGS: eventos por tipo, eventos de magnitude 4 ou mais por região e eventos por dia (contagens exatas), profundidade média por

Tabela 2: Força de cada validator contra os 23 comportamentos defeituosos reais, com um programa representante por comportamento (RQ1). “Err. barradas” é a fração das saídas erradas que o validator barrou.

Validator	Pegos	Err. barradas	Rej. falsa
V0 formato	6/23 (26,1%)	34,7%	0,0%
V1 ancorado	6/23 (26,1%)	34,7%	0,0%
V2 propriedades	21/23 (91,3%)	95,6%	0,4%
V3 metamórfico	8/23 (34,8%)	39,3%	0,0%
V2 + V3	21/23 (91,3%)	95,6%	0,4%
V4-1 N-version	18/23 (78,3%)	61,3%	8,2%
V4-2 N-version	22/23 (95,7%)	69,4%	27,2%
V4-3 N-version	15/15 (100%)	100%	10,2%

região (até 1 km) e os dez eventos mais significativos (ao menos 9 de 10). Três delas (conciliação, série de alertas e eventos por dia) exigem exatidão por natureza e foram incluídas para não escolher só tarefas tolerantes. O consumidor é tolerante como um consumidor real: lê números em texto e timestamps com milissegundos. As tarefas e tolerâncias foram congeladas com hash SHA-256 antes da primeira execução contra o corpus.

Cada programa é avaliado em dois escopos. No escopo *formato*, a tarefa é calculada só sobre os registros da assinatura do programa, com as saídas dele e com a verdade; é o consumidor daquela fonte e não depende da composição do fluxo. No escopo *fluxo*, os demais formatos do domínio entram com a verdade, e o efeito do programa se dilui. Registros que o programa rejeita somem do conjunto; registros que deveriam ser rejeitados e que ele entrega entram. Um comportamento é *inofensivo* se resolve todas as tarefas do domínio e *grave* se não resolve nenhuma.

4 Resultados

4.1 RQ1: Força contra falhas reais

A Tabela 2 mostra o resultado principal. O validator de formato (V0) pegou 6 dos 23 comportamentos defeituosos; ancorar o identificador na fonte (V1) não acrescentou nada, porque nenhum programa do corpus inventou identificadores. O validator de propriedades (V2) pegou 21 dos 23, e barrou todas as saídas erradas desses 21, com 0,4% de rejeição falsa. O metamórfico (V3) pegou 8, e combiná-lo com V2 não mudou nada. N-version pegou de 78% a 100% conforme o parceiro, mas barrou de 8,2% a 27,2% das saídas corretas: sempre que o parceiro erra, a saída correta é rejeitada. O parceiro V4-3 existe só para 15 dos 23 comportamentos, porque nem todo formato tem três programas no corpus.

Contando programas únicos em vez de comportamentos, o quadro é o mesmo: V0 pega 15,7% dos 70 programas defeituosos e V2 94,3%, com rejeição falsa de 0,1%.

A Tabela 3 organiza os 23 comportamentos. V0 só pega defeitos que quebram o formato da saída: timestamps com milissegundos fora do padrão do telos, e campos deslocados quando um CSV com vírgula dentro de aspas é partido na vírgula. Todo o resto é bem-formado: valores 100 vezes maiores, datas em 1970, fusos ignorados, moedas ou timestamps inventados para registros incompletos, campos fabricados sob o telos estrito. V2 pega esses casos porque

Tabela 3: Taxonomia dos 23 comportamentos defeituosos e quantos cada validator pega (O = só Opus 4.5, H = só Haiku 4.5, A = ambos).

Classe de defeito	Modelos	V0	V2	V3
Número lido na convenção errada (“2181.38” → 218138)	2 H	0/2	2/2	0/2
Tempo: fuso ignorado, epoch em s lido como ms	1 O, 2 H	0/3	3/3	1/3
Saída fora do formato do telos (milissegundos)	3 H	3/3	3/3	3/3
Semântica do valor (arredondamento, unidade não convertida, dupla conversão, categoria)	2 O, 1 H, 1 A	0/4	2/4	0/4
Chute em vez de rejeição (moeda ou timestamp ausente)	1 O, 2 H, 1 A	0/4	4/4	0/4
Campo fabricado sob telos insatisfatório	3 O, 2 H, 1 A	2/6	6/6	3/6
Parsing (vírgula dentro de aspas no CSV)	1 H	1/1	1/1	1/1
Total	7 O, 13 H, 3 A	6/23	21/23	8/23

verifica o que V0 não tem como verificar: se o número da saída é explicável por um número da entrada, se a data da saída corresponde a uma data da entrada, e se a entrada contém a moeda ou o timestamp que a saída afirma.

O que escapou. Os dois comportamentos que V2 não pegou estão no formato financeiro pré-agregado, em que a entrada já traz o valor em USD e a moeda original. Um programa converteu o valor em USD de novo, como se fosse BRL; o resultado continua sendo “um número da entrada vezes a taxa de uma moeda citada na entrada”, e uma relação genérica não distingue os dois casos. O outro derivou a categoria da moeda do valor (USD) em vez da moeda original. Este segundo caso expõe uma **ambiguidade da própria especificação**: o telos diz que a categoria é “domestic se a moeda é BRL”, sem dizer qual moeda vale quando o registro traz duas. Nenhum validator derivado dessa especificação pode resolver a ambiguidade; o oráculo do IDAE a resolveu por convenção.

4.2 RQ3: Validators escritos pelo próprio LLM

A Tabela 4 resume os 80 validators escritos pelos LLMs. Eles pegam muito: a mediana dos validators do Haiku 4.5 pega 88,9% dos comportamentos defeituosos do financeiro, mais que V2. Mas quase nunca são utilizáveis: a mediana da rejeição falsa do Haiku 4.5 é 90,8% no financeiro e 51,9% no IoT, e nenhum dos 20 validators do Haiku nesses dois domínios fica abaixo de 1% de rejeição falsa. O Opus 4.5 é mais permissivo e, por isso, pega menos; só 1 de 10 validators do Opus no financeiro e 3 de 10 no IoT ficam abaixo de 1%. No USGS estrito, em que basta exigir o campo sig, todos os 20 são utilizáveis, mas a mediana do Opus pega só 33,3%.

Com autoverificação iterativa. A revisão consumiu, em média, 3,3 rodadas por validator (185 chamadas de revisão) e funcionou *nos exemplos*: no financeiro, os validators do Opus 4.5 que aceitam todas as saídas corretas dos exemplos passaram de 2 para 6 em 10, e a fração de variantes erradas aceitas caiu de 22,6% para

Tabela 4: Validators escritos por LLM em uma chamada (L) e os mesmos após autoverificação iterativa (I), com 10 validators por modelo e domínio (RQ3). “Pega” é a mediana da fração de comportamentos defeituosos pegos e “Rej. falsa”, a mediana da rejeição falsa; “Utiliz.” conta os validators com rejeição falsa de até 1%.

Domínio	Validator	Pega	Rej. falsa	Utiliz.
Financeiro	V2	77,8%	0,0%	—
	L Opus 4.5	66,7%	9,5%	1/10
	L Haiku 4.5	88,9%	90,8%	0/10
	I Opus 4.5	77,8%	9,2%	0/10
	I Haiku 4.5	88,9%	89,1%	0/10
IoT	V2	100%	1,2%	—
	L Opus 4.5	71,4%	2,7%	3/10
	L Haiku 4.5	85,7%	51,9%	0/10
	I Opus 4.5	78,6%	4,2%	4/10
	I Haiku 4.5	85,7%	51,9%	0/10
USGS corrigido	V2	100%	0,0%	—
	L Opus 4.5	100%	0,0%	6/10
	L Haiku 4.5	100%	59,3%	4/10
	I Opus 4.5	100%	0,0%	7/10
	I Haiku 4.5	100%	0,0%	6/10
USGS estrito	V2	100%	0,0%	—
	L Opus 4.5	33,3%	0,0%	10/10
	L Haiku 4.5	100%	0,0%	10/10
	I Opus 4.5	33,3%	0,0%	10/10
	I Haiku 4.5	100%	0,0%	10/10

0,1%. Contra as falhas reais, quase nada mudou (Tabela 4, linhas I). A mediana pega um pouco mais no financeiro (66,7% → 77,8%) e no IoT (71,4% → 78,6%) com o Opus, mas a rejeição falsa continua alta: nos domínios financeiro e IoT, 4 de 40 validators eram utilizáveis antes e 4 de 40 depois. Os do Haiku 4.5 no financeiro continuaram rejeitando saídas corretas dos próprios exemplos depois de cinco rodadas. Só no USGS corrigido a revisão resolveu a rejeição falsa do Haiku (mediana de 59,3% para 0,0%). Pareando cada validator com sua revisão, 16 dos 80 passaram a pegar mais comportamentos e 3 menos; 15 reduziram a rejeição falsa e 13 a aumentaram.

O motivo é sobreajuste ao exemplo. Os seis validators do Opus 4.5 no financeiro que passaram limpos na autoverificação rejeitaram, no fluxo, 11,3% das saídas corretas, quase sempre em registros normais dos mesmos formatos dos exemplos. No formato chave=valor, por exemplo, um deles rejeita todos os registros cujo horário, convertido para UTC, cai no dia seguinte, e aceita todos os demais: o único exemplo desse formato não cruza o dia, e a autoverificação não tem como revelar a regra errada. V2 teve os mesmos exemplos e não caiu nisso. Uma vez que a iteração com os mesmos exemplos não reproduz V2, a diferença que resta entre V2 e L é o modelo e a liberdade do agente para escrever os próprios testes e raciocinar sobre a especificação, não a possibilidade de iterar.

Erros correlacionados. Os validators do Opus 4.5 aceitaram 69,6% das saídas erradas de programas do Opus e 20,3% das do Haiku; os do Haiku 4.5, 29,3% e 11,3%. Os dois modelos aceitam mais os erros do Opus, que são mais sutis, e o Opus é mais permissivo em geral. Controlando as duas coisas, a razão de chances da

Tabela 5: Escore de mutação sobre 265 mutantes com comportamento distinto, comparado com a força contra falhas reais (RQ2).

Validator	Mutantes pegos	Rej. falsa	Falhas reais pegas
V0 formato	30,9%	0,0%	26,1%
V1 ancorado	57,4%	0,0%	26,1%
V2 propriedades	99,6%	2,2%	91,3%
V3 metamórfico	55,1%	9,7%	34,8%
V4-1 N-version	100%	2,0%	78,3%

interação (quanto cada modelo é relativamente mais leniente com os próprios erros) é 2,76, com intervalo de 95% de [0,28; 18,2] por bootstrap sobre os 23 comportamentos. Com os validators revisados (I), a razão é 5,75 [0,72; 96,8]. A direção é compatível com erros correlacionados, como no N-version clássico [16], mas o dado não permite concluir.

4.3 RQ2: Escore de mutação como medida barata

A Tabela 5 mostra o escore de mutação dos validators de referência. A ordenação acompanha a das falhas reais nos extremos (V0 fraco, V2 forte), mas não no meio: V1 pega 57,4% dos mutantes e nenhuma falha real a mais que V0, porque os operadores numéricos alteram também dígitos de identificadores, algo que os LLMs do corpus não fizeram. N-version chega a 100% por construção, já que o parceiro é um programa correto do qual o mutante difere; o escore de mutação superestima essa estratégia.

Sobre todos os validators (os quatro de referência e os 20 de LLM por domínio, um ponto por domínio e validator, N-version excluído), a correlação de Spearman entre o escore de mutação e a fração de comportamentos reais pegos é $\rho = 0,51$ (IC 95% por bootstrap [0,36; 0,66], $n = 100$); pela fração de saídas erradas baratas, $\rho = 0,62$ [0,48; 0,75]. Por domínio, $\rho = 0,81$ no financeiro, 0,87 no USGS estrito e 0,46 [0,04; 0,78] no IoT; no USGS corrigido não há variância, porque todos os validators pegam o único comportamento defeituoso. O escore de mutação serve, portanto, como triagem para descartar validators fracos antes do deploy, mas não substitui um teste com falhas reais, em linha com o que se sabe sobre mutantes como substitutos de falhas reais em teste de software [15].

4.4 RQ4: Custo

A Tabela 6 resume o custo por verificação, medido em saídas corretas de programas corretos (Node 22). V0 custa de 1 a 2 μ s. V2 custa de 50 a 110 μ s de mediana e não executa o programa de novo. V3 custa de 0,63 a 0,73 ms, porque reexecuta o programa cinco ou seis vezes por registro. V4 exige uma síntese extra por formato (uma chamada de LLM) e uma execução extra por registro. Os validators de LLM custam de 105 a 120 μ s por verificação; para criar, uma chamada (L) ou, com autoverificação, 3,3 chamadas em média (I). O módulo de V2 e V3 tem de 276 a 301 linhas por domínio, contra 6 a 12 de V0 e medianas de 39 a 88 (L) e de 45 a 89 (I) dos validators de LLM. Os tempos são de uma única execução e variam entre execuções; a ordem de grandeza é estável.

Tabela 6: Custo de cada estratégia: tempo por verificação (mediana em μ s, faixa entre os domínios) e o que é preciso para criar o validator (RQ4).

Validator	μ s/verif.	Exec. extras	Para criar
V0 formato	1–2	0	manual (6–12 linhas)
V2 propriedades	50–110	0	agente (276–301 linhas)
V3 metamórfico	630–730	5–5,7	idem (mesmo módulo)
V4 N-version	140–190	1	1 síntese por formato
L (LLM)	105–115	0	1 chamada
I (LLM iterado)	105–120	0	3,3 chamadas (média)

Tabela 7: Fração dos 23 comportamentos defeituosos pegos por um portão de k pares verificados por formato, sozinho (G) e combinado com V0 e V2 (RQ5). Para sorteio e estratificado, valores esperados. Nenhuma configuração barrou um programa correto.

Composição	k	G	G + V0	G + V2
V2 sozinho	0	—	—	91,3%
Primeiros	1	78,3%	82,6%	100%
	10	82,6%	87,0%	100%
Sorteio	1	65,7%	71,4%	95,7%
	3	77,4%	81,8%	98,1%
	10	85,9%	90,0%	99,8%
Estratificado	1	61,7%	69,4%	95,7%
	3	86,2%	90,5%	98,1%
	10	95,1%	99,2%	99,8%

4.5 RQ5: Pares verificados

A Tabela 7 mostra o resultado. Nenhuma composição barrou um programa correto (0 de 27 comportamentos corretos e 0 de 961 programas corretos): um par verificado só reprova quem erra nele. Os pares também barram programas incompletos (que devolvem null para registros válidos): 14 dos 15 comportamentos incompletos com os 10 primeiros pares.

Sozinhos, os pares verificados pegam muito com pouco: o primeiro registro de cada formato, verificado, já pega 78,3% dos comportamentos defeituosos, porque a maioria dos defeitos aparece em quase todo registro (valores 100 vezes maiores, datas de 1970, fusos ignorados) e porque, nos formatos que só existem para ser rejeitados, o primeiro registro já é um par de rejeição. Mas estacionam: com 10 pares sorteados, 85,9%. Os defeitos que escapam são os que só aparecem em casos de borda ou em registros raros (1–2%). Um programa que inventa o timestamp ausente, por exemplo, erra em 4% dos registros do seu formato, todos casos de borda. A composição estratificada, que inclui um caso de cada tipo, sobe a 95,1% com 10 pares.

Complementaridade. Pares verificados e V2 falham em defeitos diferentes. Os dois comportamentos que escaparam de V2, a dupla conversão e a categoria derivada da moeda errada, erram em 75% e 25% dos registros do seu formato, inclusive nos primeiros, e caem no primeiro par verificado: o par carrega a convenção que

Tabela 8: Comportamentos defeituosos que cada tarefa tolera (RQ6), no escopo formato (só a fonte afetada) e no escopo fluxo (domínio inteiro, demais formatos corretos). * = tarefa que exige exatidão por natureza. O USGS soma o telos corrigido (1 comportamento) e o estrito (6).

Tarefa	Formato	Fluxo
<i>Financeiro (9 comportamentos)</i>		
Total por mês	2	3
Total por categoria	2	2
Fração internacional	5	8
Dez maiores transações	5	5
Conciliação diária*	1	1
<i>IoT (7 comportamentos)</i>		
Média por métrica, zona e mês	3	3
Alertas críticos por mês	5	5
Distribuição métrica–status	4	6
Série de alertas críticos*	4	4
<i>USGS (7 comportamentos)</i>		
Eventos por tipo	0	0
Magnitude ≥ 4 por região	2	2
Profundidade média por região	0	0
Dez mais significativos	2	7
Eventos por dia*	0	0

a especificação não explícita. Os que escapam dos pares são justamente os de rejeição e os raros, que V2 pega por exigir evidência na entrada. Juntos, pegam de 95,7% (um par sorteado) a 100% (os primeiros registros) dos comportamentos defeituosos, e 97,9% a 100% dos 70 programas defeituosos, com a rejeição falsa de V2 (0,5% das saídas corretas dos programas aprovados). Com V0 no lugar de V2, a combinação fica em 69–99%: sem as relações, os pares precisam cobrir os casos de borda.

4.6 RQ6: Impacto na tarefa

Os 961 programas corretos resolvem todas as 14 tarefas nos dois escopos, o que serve de controle. Dos 23 comportamentos defeituosos, no escopo formato, só 3 são inofensivos, 12 resolvem algumas tarefas e não outras, e 8 não resolvem nenhuma; contando programas, 5, 46 e 19 dos 70. Os três inofensivos são o mesmo defeito: timestamps do IoT com milissegundos fora do padrão do telos, que nenhum consumidor tolerante percebe. Um quarto comportamento, a data errada em 20% das transações de um formato, resolve todas as tarefas menos a conciliação diária.

O validator de formato inverte as prioridades. Dos 6 comportamentos que V0 pega, 3 são os inofensivos; dos 17 que ele deixa passar, nenhum é inofensivo, e todos mudam ao menos uma tarefa. V0 rejeita o que não importa para quem usa os dados e aceita o que importa. Todos os portões avaliados rejeitam os três inofensivos, inclusive o oráculo exato, de modo que exigir exatidão custou só três ressinteses. O que os separa é o dano aceito entre os 20 comportamentos restantes: V0 aceita 17, os pares verificados sozinhos 5, V2 2, e V2 com o primeiro par verificado, nenhum.

Diluir ajuda as proporções, não os totais. No escopo fluxo, os outros formatos do domínio entram corretos e quase todo comportamento passa a resolver alguma tarefa (19 de 23 ficam no meio,

1 continua sem resolver nenhuma), mas os mesmos três continuam sendo os únicos inofensivos. A Tabela 8 mostra onde a diluição ajuda: proporções e rankings (fração internacional, distribuição de status, eventos mais significativos) passam a tolerar quase todos os defeitos, porque dependem pouco do campo errado ou porque o erro some na massa. Totais, médias e contagens exatas continuam quebrando. O programa que multiplica valores por 100 erra o total mensal do domínio inteiro em 1.549%, e a dupla conversão, em 3,2%, acima da tolerância de 1%.

5 Discussão: o que um validator precisa checar

Relacione a saída com a entrada, não só com o formato. A maior parte das falhas reais é bem-formada. O que as denuncia é a relação com a entrada: o número da saída não é explicável por nenhum número da entrada sob as conversões permitidas; a data da saída não está na entrada; a entrada não contém a moeda ou o timestamp que a saída afirma. Essas relações podem ser escritas de forma genérica, tratando a entrada como texto, sem depender de nomes de campos ou delimitadores. A instância que escreveu V2 recebeu um exemplo de cada formato avaliado, inclusive dos novos; se as relações continuam valendo para formatos nunca vistos é uma hipótese que este estudo não testa.

Exija evidência para o que a especificação manda rejeitar. Quatro dos 23 comportamentos são programas que “chutam” um valor para um dado ausente. Um validator que exige, na entrada, evidência de cada campo obrigatório converte esse chute em rejeição.

Metamórfico e N-version não compensam aqui. V3 custou de seis a quinze vezes mais que V2 e não acrescentou nada sobre ele. N-version pegou muito, mas com rejeição falsa alta e instável: a qualidade do portão passa a depender de qual parceiro foi sorteado.

Não delegue o validator ao mesmo processo que gera o programa. Um validator gerado pelo LLM numa única chamada tende a ser ou permissivo demais ou restritivo demais, e deixar o mesmo modelo revisá-lo contra os exemplos da especificação não resolve: ele passa nos exemplos e continua rejeitando registros corretos que os exemplos não cobrem. Uma autoverificação com um exemplo por formato mede pouco. Se o validator for gerado por LLM, ele precisa ser validado contra um conjunto de registros corretos bem maior e mais variado que o do prompt, por exemplo uma amostra de produção revisada, antes de virar portão.

A especificação é o teto do validator; exemplos verificados passam dele. Os dois defeitos que escaparam de V2 não são falhas do validator, mas limites da especificação: uma conversão plausível que a especificação não distingue e uma ambiguidade sobre qual moeda define a categoria. Um validator derivado da especificação não passa desse teto; um par verificado passa, porque carrega a convenção que a especificação omite.

A amortização transforma defeito em viés. Um programa descartável errado não erra “às vezes, para cima e para baixo”: erra do mesmo jeito em todos os registros do seu formato. Para quem consome os dados de forma estatística, isso é viés sistemático, que nenhuma média cancela; ele só se dilui na proporção da fração dos dados que vem daquele formato. Um defeito só é tolerável quando a estatística consumida não depende do campo errado ou quando

essa fração vezes o tamanho do erro cabe na tolerância. Como res-sintetizar custa uma chamada de LLM e, em 13 dos 15 formatos com defeito, o corpus tem ao menos um programa correto, aceitar um programa sabidamente impreciso raramente compensa. As exceções são os outros dois formatos, para os quais nenhum programa do corpus acertou (o CSV do USGS estrito, que deveria ser rejeitado por inteiro, e o formato financeiro chave=valor com barras verticais), e um modo provisório enquanto um programa novo aguarda aprovação.

Comece cada programa por pares verificados. Se cada formato tiver poucos pares com a saída verificada, incluindo casos de borda e entradas que devem ser rejeitadas, e o formato for estável durante o uso, exigir que o programa os reproduza antes de aprová-lo não barra nenhum programa correto e, combinado com um validator de relações, pegou praticamente todos os defeitos do corpus. O custo é humano: verificar alguns registros por formato novo. A escolha dos pares importa mais que a quantidade: um caso de cada tipo rende mais que muitos casos normais.

6 Ameaças à Validade

Autoria de V2. V2 e V3 foram escritos por uma instância do Claude Opus 5.5, modelo mais recente que os estudados, trabalhando como agente: derivou as saídas corretas dos exemplos, escreveu os próprios testes e iterou. Os validators L foram gerados em uma única chamada por Opus 4.5 e Haiku 4.5, e os I revisados pelos mesmos modelos com autoverificação automática sobre os mesmos exemplos. O X5 mostra que a iteração, sozinha, não explica a diferença; ela fica atribuída ao modelo e à liberdade do agente, que este estudo não separa. A comparação não deve ser lida como “humano contra LLM”. A instância que escreveu V2 não teve acesso ao corpus, aos defeitos ou aos resultados, e o código foi congelado antes da avaliação; não é possível, porém, excluir que o conhecimento geral do modelo sobre defeitos comuns de parsing tenha influenciado as relações escolhidas.

Feedback do X5. O mutador de saídas e o limite de cinco rodadas são escolhas nossas; um feedback mais rico (mais exemplos corretos por formato, entradas que devem ser rejeitadas) pode dar outro resultado. Em particular, o feedback não contém entradas a rejeitar, o que limita o que a revisão pode aprender sobre a regra de rejeição (USGS estrito).

Conhecimento de formato. Apesar da restrição de tratar a entrada como texto opaco, o V2 do USGS contém algum conhecimento genérico de GeoJSON (ignora “Feature” e “Point” como tipo de evento e o título do evento como local).

Tamanho e origem do corpus. São 23 comportamentos defeituosos em três domínios de extração, de dois modelos da mesma família. A correlação da RQ3 é inconclusiva por falta de dado, e a generalização para outros tipos de programa é hipótese. Os formatos novos foram desenhados pelo autor depois de ver os defeitos do corpus original, o que pode ter favorecido certos tipos de defeito; os rótulos vêm do oráculo, não de julgamento do autor.

Oráculo. O oráculo do formato pré-agregado resolve por convenção uma ambiguidade da especificação; reportamos o caso como limite da especificação, não como defeito puro.

Mutação. Os operadores são textuais; alguns caem em identificadores ou literais que não correspondem a erros plausíveis de

LLM, o que explica parte da divergência de V1 entre mutantes e falhas reais. O escore de N-version é otimista por construção.

Pares verificados. A RQ5 supõe verificação perfeita (a verdade vem do oráculo) e formato estável. A composição “primeiros” reflete a ordem do nosso fluxo sintético, em que os registros normais vêm antes dos casos de borda; a composição por sorteio é a estimativa menos dependente dessa ordem. A estratificada usa os tipos de registro do nosso gerador; na prática, depende de quem verifica conhecer os casos de borda do formato.

Tarefas de consumo. As 14 tarefas e tolerâncias são escolhas do autor; outras tarefas dariam outros números. Para reduzir o viés de escolha, elas foram congeladas antes da execução e incluem três que exigem exatidão. Depois do congelamento houve uma correção: a comparação dos dez maiores falhava quando o formato tinha menos de dez registros válidos, o que o controle com programas corretos revelou; a tolerância declarada não mudou. A RQ6 mede impacto em tarefas agregadas, não o que uma pessoa considera aceitável.

Representante por comportamento. Programas com o mesmo comportamento podem dar saídas erradas diferentes; reportamos o primeiro programa de cada grupo e, em separado, os resultados por programa único, que levam às mesmas conclusões.

7 Trabalhos Relacionados

Oráculos de teste. O problema de decidir se uma saída está correta sem conhecer a resposta é central em teste de software [2]. Relações metamórficas [4, 20] e propriedades [7] são as formas clássicas de oráculo parcial; aqui elas são usadas em produção, como portão de aceitação de programas sintetizados. Invariantes também podem ser inferidos de execuções observadas, como no Daikon [10]; V2, ao contrário, é derivado da especificação, o que o protege de aprender como invariante o comportamento de um programa defeituoso.

N-version e independência de falhas. A programação N-version [1] supõe falhas independentes, suposição refutada experimentalmente [16]. Trabalhos recentes geram as variantes com LLMs [19]; nossos resultados mostram o custo dessa estratégia em rejeição falsa quando usada como portão.

Oráculos gerados por modelos. Oráculos de teste gerados por modelos neurais [8] tiveram, numa avaliação em larga escala, mais de 47% de falsos positivos entre as asserções geradas [12], o equivalente, em teste, à rejeição falsa que medimos. Pós-condições geradas por LLMs a partir de linguagem natural discriminam código errado e pegaram falhas históricas reais [9]; nossos validators L são o análogo em produção, com a diferença de que medimos também quantas saídas corretas eles barram. Oráculos gerados por LLMs tendem a capturar o comportamento que o programa tem, e não o esperado [17], o que motiva a pergunta sobre erros correlacionados da RQ3. Uma visão geral do uso de LLMs em teste está em [21].

LLMs verificando LLMs. Gerar testes com o próprio modelo para escolher entre soluções [5] e usar LLMs como juizes [22] são práticas difundidas. Nossos dados sugerem que, como portão de aceitação, o validator gerado pelo LLM precisa ser validado antes do uso.

Autocorreção. LLMs corrigem o próprio código quando recebem resultados de execução [6], mas não melhoram de forma confiável sem feedback externo [13]. O X5 fica entre os dois: há feedback externo, mas restrito aos exemplos da especificação, e a melhora nos exemplos não se transferiu para o fluxo.

Exemplos como especificação. Pares entrada-saída são a especificação na programação por exemplos [11], em que servem para *synetizar* o programa. Na RQ5, os mesmos pares servem para *aceitar* um programa sintetizado a partir de uma especificação em linguagem natural, e cobrem justamente o que essa especificação deixa ambíguo.

Mutação como substituto de falhas reais. A validade de mutantes como substitutos de falhas reais foi estudada para suítes de teste [14, 15]; estendemos a pergunta para validadores de programas sintetizados por LLM.

IDAE. Este artigo parte da avaliação do IDAE [3], que mostrou que a garantia da arquitetura é a força da sua especificação, e reutiliza seus domínios, oráculos e logs.

8 Considerações Finais

Um validator de formato pegou 6 de 23 comportamentos defeituosos reais de programas sintetizados por LLM. Um validator que relaciona cada saída com a entrada bruta, escrito só a partir da especificação e sem ver as falhas, pegou 21, com 0,4% de rejeição falsa, a um custo de dezenas de microssegundos por registro. Validadores escritos pelos próprios LLMs foram fortes, mas quase sempre inutilizáveis por rejeitarem saídas corretas, mesmo depois de revisados pelo próprio modelo contra os exemplos da especificação; N-version foi forte e instável. Pares verificados por formato, exigidos antes de aprovar cada programa, não barraram nenhum programa correto e pegaram exatamente os defeitos que o validator de relações deixou passar; juntos, pegaram de 96% a 100% dos comportamentos defeituosos. O escore de mutação correlacionou moderadamente com a força real e serve como triagem. Contra tarefas de consumo fixadas antes da análise, 20 dos 23 comportamentos defeituosos mudaram o resultado de alguma tarefa, e o validator de formato pegou justamente os três que não mudavam: a amortização faz de cada defeito um viés sistemático, que o uso estatístico não cancela. E os dois defeitos que escaparam do validator mostram o seu teto: a própria especificação, que só exemplos verificadas ultrapassam.

Trabalhos futuros incluem ampliar o corpus para outros tipos de programa e mais famílias de modelos, separar o efeito do modelo do efeito do processo de agente na escrita de validadores, medir se uma autoverificação contra uma amostra maior de registros corretos torna utilizáveis os validadores gerados por LLM, e aplicar a mesma régua a domínios em que a especificação é um conjunto de invariantes, como mecânicas geradas em jogos.

O próximo passo natural, e o que pretendemos construir, é um software de *aprovação humana* para esses pontos variáveis. Quando o validator ou os pares verificados invalidam um programa descartável, os payloads que chegam daquele formato ficam retidos em vez de processados ou descartados; o sistema sintetiza um programa novo e o apresenta a uma pessoa com a evidência para decidir: o registro que invalidou o anterior, a saída do novo programa nos pares verificados e nos payloads retidos, e

as relações que o validator checkou. Aprovado, o programa entra em uso e processa os payloads retidos; cada saída conferida na aprovação vira mais um par verificado do formato. A hipótese é que, em trechos de software que mudam com frequência e já são mantidos com IA, esse ciclo de aprovação pode substituir o ciclo de QA e deploy: o que precisa de validação humana é o programa novo e as saídas que ele produz, não uma nova versão do sistema. Os resultados deste artigo sugerem o que mostrar a quem aprova (pares verificados e relações com a entrada, não a opinião de outro LLM), mas deixam abertas as perguntas que um estudo com usuários teria de responder: quanto tempo os payloads podem ficar retidos, com que frequência a aprovação é necessária, e se quem aprova, diante de evidência favorável, passa a aprovar sem conferir.

Disponibilidade de Artefatos

Os artefatos estão disponíveis em <https://github.com/carloseduardoddb/idae-validators>. O pacote contém o corpus de programas (com código, rótulos e proveniência de cada resposta), os logs JSONL de todas as chamadas de LLM, as fases novas dos domínios, os validadores V2/V3 congelados com hashes SHA-256 e o material que a instância que os escreveu recebeu, os 80 validadores de LLM congelados e suas versões revisadas (com todas as rodadas), os mutantes, os scripts dos experimentos X0–X7, as tarefas de consumo congeladas e os resultados brutos. O artigo do IDAE está arquivado no Zenodo sob o DOI 10.5281/zenodo.23050791 (versão em inglês: 10.5281/zenodo.23062292), e seu código em <https://github.com/carloseduardoddb/idae>.

Declaração de Uso de IA Generativa

Além do uso de LLMs como objeto de estudo (Claude Opus 4.5 e Haiku 4.5 como sintetizadores de programas e de validadores), foi usado um assistente de IA generativa (Claude Opus 5.5) para: desenhar parte dos experimentos, implementar os scripts de avaliação e as análises estatísticas, buscar e conferir referências e redigir o texto do artigo. Numa instância isolada, descrita na Seção 3, o mesmo assistente escreveu os validadores V2 e V3. O autor definiu as perguntas de pesquisa, revisou o código, os resultados e o texto, e assume responsabilidade integral pelo conteúdo.

Referências

- [1] Algirdas Avižienis. 1985. The N-version approach to fault-tolerant software. *IEEE Transactions on Software Engineering* SE-11, 12 (1985), 1491–1501.
- [2] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering* 41, 5 (2015), 507–525.
- [3] Carlos Eduardo Dias Batista. 2026. Programas descartáveis sob especificação imutável: o que um validator realmente garante — um estudo com extração de dados. Zenodo. <https://doi.org/10.5281/zenodo.23050791>
- [4] Tsong Yueh Chen, Shing Chi Cheung, and Siu Ming Yiu. 1998. *Metamorphic Testing: A New Approach for Generating Next Test Cases*. Technical Report HKUST-CS98-01. Hong Kong University of Science and Technology.
- [5] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. CodeT: Code generation with generated tests. arXiv:2207.10397.
- [6] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching large language models to self-debug. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. arXiv:2304.05128.
- [7] Koen Claessen and John Hughes. 2000. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 268–279.

- [8] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K. Lahiri. 2022. TOGA: A neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. <https://doi.org/10.1145/3510003.3510141>
- [9] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. 2024. Can large language models transform natural language intent into formal method postconditions? *Proceedings of the ACM on Software Engineering* 1, FSE (2024). <https://doi.org/10.1145/3660791>
- [10] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. 2007. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming* 69 (2007), 35–45.
- [11] Sumit Gulwani. 2011. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 317–330.
- [12] Soneya Binta Hossain, Antonio Filieri, Matthew B. Dwyer, Sebastian Elbaum, and Willem Visser. 2023. Neural-based test oracle generation: A large-scale evaluation and lessons learned. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. <https://doi.org/10.1145/3611643.3616265>
- [13] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. arXiv:2310.01798.
- [14] Yue Jia and Mark Harman. 2011. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering* 37, 5 (2011), 649–678.
- [15] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. 2014. Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 654–665.
- [16] John C. Knight and Nancy G. Leveson. 1986. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 96–109.
- [17] Michael Konstantinou, Renzo Degiovanni, and Mike Papadakis. 2024. Do LLMs generate test oracles that capture the actual or the expected program behaviour? arXiv:2410.21136.
- [18] Bertrand Meyer. 1997. *Object-Oriented Software Construction* (2nd ed.). Prentice Hall.
- [19] Javier Ron, Diogo Gaspar, Javier Cabrera-Arteaga, Benoit Baudry, and Martin Monperrus. 2024. Galápagos: Automated N-version programming with LLMs. arXiv:2408.09536.
- [20] Sergio Segura, Gordon Fraser, Ana B. Sanchez, and Antonio Ruiz-Cortés. 2016. A survey on metamorphic testing. *IEEE Transactions on Software Engineering* 42, 9 (2016), 805–824.
- [21] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* 50, 4 (2024), 911–936.
- [22] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.